

OzZoo Simulation Game - Technical Design Document

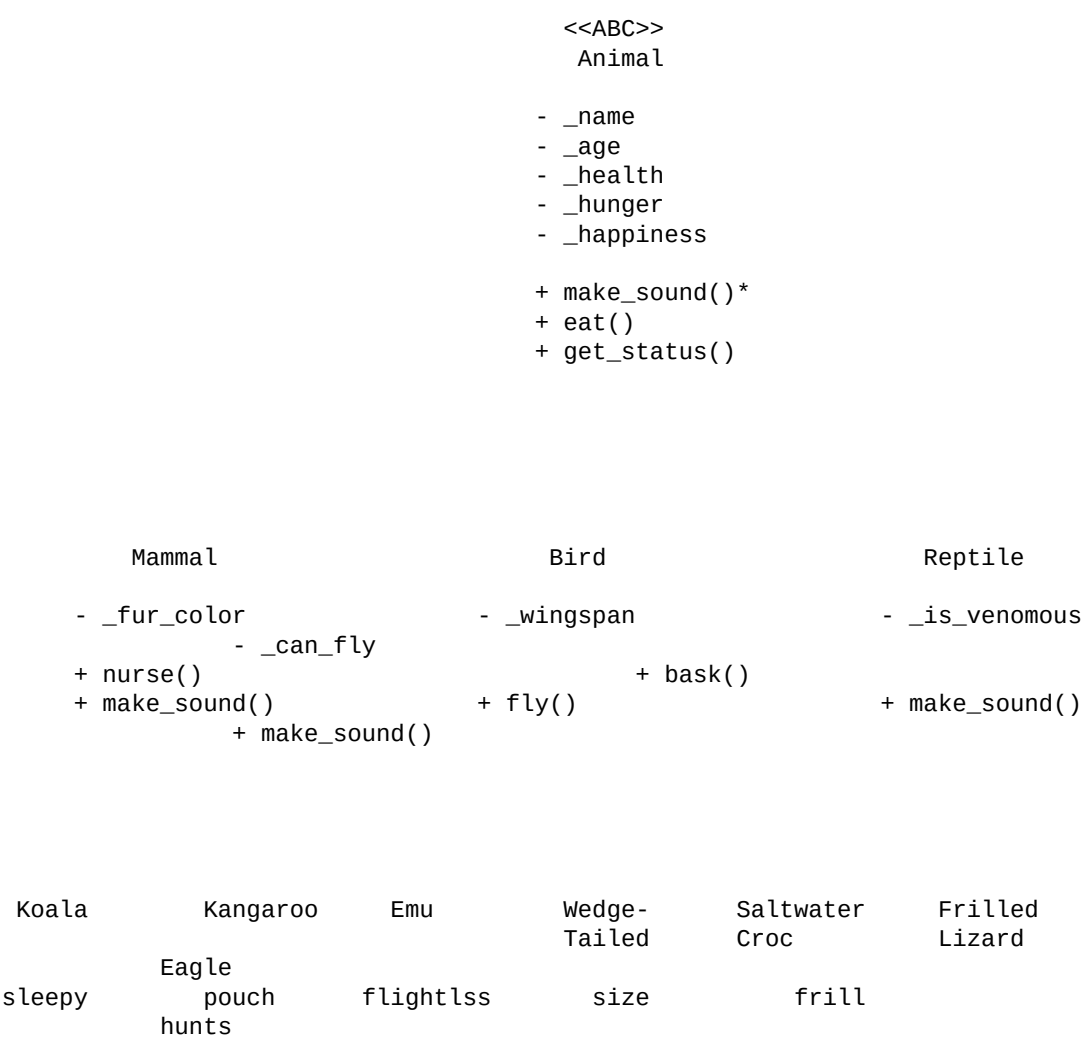
1. Project Overview

****Project:**** OzZoo - Australian Wildlife Park Simulation
****Assessment:**** Project (30%) + Presentation (20%) = 50% Total
****Theme:**** Manage a wildlife park featuring Australian native animals

Background Story

G'day, Manager! Welcome to OzZoo, a wildlife park on the outskirts of the city. Your goal is to turn OzZoo into a world-class centre for animal conservation, education, and visitor enjoyment while balancing finances, animal welfare, and visitor satisfaction.

2. Class Hierarchy Diagram



Additional Classes:

Wombat (Mammal)	Platypus (Mammal)	Tasmanian Devil	Echidna (Mammal)
burrows	venomous	spines	
spur	loud_screech		

<<ABC>>
Enclosure

```

- _enclosure_id
- _name
- _capacity
- _animals[]
- _cleanliness

+ add_animal()
+ clean()*
+ get_status()

```

Eucalyptus Grove	Outback Savanna	Billabong	Rainforest Aviary	Reptile House
tree_count For: Koalas	shade_areas For: Roos Wombats	water_level For: Crocs, Platypus	humidity For: Birds	temperature For: Lizards

Zoo	Visitor	ZooKeeper
<pre> - _name - _budget - _enclosures[] - _animals[] - _visitors[] - _day - _food_stock{} + donate() + advance_day() + add_animal() + buy_food() + get_report() </pre>	<pre> - _visitor_id - _name - _ticket_type - _satisfaction + view_animal() + buy_snack() + heal_animal() </pre>	<pre> - _employee_id - _name - _specialty - _assigned_enc + feed_animals() + clean_encl() </pre>
	Food	Medicine
	<pre> - _name - _type - _cost - _nutrition + apply() + __str__() </pre>	<pre> - _name - _treats - _cost </pre>

3. Interface Definitions

ICleanable Interface (ABC)

```
from abc import ABC, abstractmethod
```

```

class ICleanable(ABC):
    """Interface for objects that can be cleaned."""

    @abstractmethod
    def clean(self) -> None:
        """Clean the object, improving its cleanliness level."""
        pass

    @abstractmethod
    def get_cleanliness(self) -> int:
        """Return current cleanliness level (0-100)."""
        pass

```

IFeetable Interface (ABC)

```

class IFeedable(ABC):
    """Interface for entities that can be fed."""

    @abstractmethod
    def feed(self, food: 'Food') -> bool:
        """Feed the entity. Returns True if successful."""
        pass

    @abstractmethod
    def get_diet(self) -> str:
        """Return the required diet type."""
        pass

    @abstractmethod
    def is_hungry(self) -> bool:
        """Check if the entity is hungry."""
        pass

```

IObservable Interface (for Observer Pattern)

```

class IObservable(ABC):
    """Interface for observable subjects in Observer pattern."""

    @abstractmethod
    def attach(self, observer: 'IObserver') -> None:
        """Attach an observer."""
        pass

    @abstractmethod
    def detach(self, observer: 'IObserver') -> None:
        """Detach an observer."""
        pass

    @abstractmethod
    def notify(self, event: str) -> None:
        """Notify all observers of an event."""
        pass

```

4. Custom Exception Hierarchy

```

# =====
# CUSTOM EXCEPTIONS
# =====

class ZooError(Exception):
    """Base exception for all zoo-related errors."""
    pass

class AnimalError(ZooError):
    """Base exception for animal-related errors."""
    pass

class IncompatibleSpeciesError(AnimalError):
    """Raised when incompatible species are placed together."""
    def __init__(self, animal1: str, animal2: str, enclosure: str):
        self.message = f"Cannot place {animal1} with {animal2} in {enclosure} - incompatible species!"
        super().__init__(self.message)

class AnimalSickError(AnimalError):
    """Raised when an animal is too sick for an action."""
    def __init__(self, animal_name: str, action: str):
        self.message = f"{animal_name} is too sick to {action}!"
        super().__init__(self.message)

class ResourceError(ZooError):
    """Base exception for resource-related errors."""
    pass

```

```

class InsufficientFoodError(ResourceError):
    """Raised when there isn't enough food."""
    def __init__(self, food_type: str, required: int, available: int):
        self.message = f"Insufficient {food_type}: need {required}, have {available}"
        super().__init__(self.message)

class InsufficientFundsError(ResourceError):
    """Raised when budget is insufficient."""
    def __init__(self, cost: float, budget: float):
        self.message = f"Insufficient funds: need ${cost:.2f}, have ${budget:.2f}"
        super().__init__(self.message)

class EnclosureError(ZooError):
    """Base exception for enclosure-related errors."""
    pass

class HabitatCapacityExceededError(EnclosureError):
    """Raised when enclosure capacity is exceeded."""
    def __init__(self, enclosure_name: str, capacity: int):
        self.message = f"{enclosure_name} is at full capacity ({capacity} animals)!"
        super().__init__(self.message)

class InvalidHabitatError(EnclosureError):
    """Raised when animal is placed in wrong habitat type."""
    def __init__(self, animal: str, enclosure: str, required: str):
        self.message = f"{animal} cannot live in {enclosure} - requires {required} habitat"
        super().__init__(self.message)

```

5. Design Pattern: Factory Pattern

The **Factory Pattern** is recommended for this project as it elegantly handles the creation of different animal types.

Why Factory Pattern?

1. **Centralizes object creation** - All animal creation goes through one place
2. **Easy to extend** - Adding new animal types only requires updating the factory
3. **Encapsulates complexity** - Client code doesn't need to know specific class constructors
4. **Validates input** - Factory can validate parameters before creating objects

Implementation

```

class AnimalFactory:
    """
    Factory for creating different types of Australian animals.

    Usage:
        factory = AnimalFactory()
        koala = factory.create_animal("koala", name="Blinky", age=3)
        kangaroo = factory.create_animal("kangaroo", name="Skippy", age=5)
    """

    # Animal type to class mapping
    _animal_types = {
        "koala": Koala,
        "kangaroo": Kangaroo,
        "wombat": Wombat,
        "platypus": Platypus,
        "tasmanian_devil": TasmanianDevil,
        "echidna": Echidna,
        "emu": Emu,
        "wedge_tailed_eagle": WedgeTailedEagle,
        "saltwater_croc": SaltwaterCroc,
        "frilled_lizard": FrilledLizard,
    }

    @classmethod

```

```

def create_animal(cls, animal_type: str, **kwargs) -> Animal:
    """
    Create an animal of the specified type.

    Args:
        animal_type: Type of animal (e.g., "koala", "kangaroo")
        **kwargs: Animal-specific attributes (name, age, etc.)

    Returns:
        Animal: The created animal instance

    Raises:
        ValueError: If animal_type is unknown
        KeyError: If required kwargs are missing
    """
    animal_type = animal_type.lower().replace(" ", "_")

    if animal_type not in cls._animal_types:
        available = ", ".join(cls._animal_types.keys())
        raise ValueError(f"Unknown animal type: '{animal_type}'. Available: {available}")

    animal_class = cls._animal_types[animal_type]

    try:
        return animal_class(**kwargs)
    except TypeError as e:
        raise KeyError(f"Missing required parameter for {animal_type}: {e}")

    @classmethod
    def get_available_types(cls) -> list:
        """Return list of available animal types."""
        return list(cls._animal_types.keys())

    @classmethod
    def register_type(cls, type_name: str, animal_class: type) -> None:
        """Register a new animal type (for extensibility)."""
        cls._animal_types[type_name.lower()] = animal_class

```

6. Game Mechanics

Day Cycle Simulation

DAY CYCLE

MORNING (Start of Day)

Animals wake up
 Hunger increases (+10)
 Visitors arrive
 Daily report displayed

PLAYER ACTIONS (Main Phase)

View zoo status
 Feed animals
 Clean enclosures
 Buy food/medicine
 Purchase new animals
 Build/upgrade enclosures
 Set ticket prices
 Treat sick animals

AFTERNOON (Mid-Day Events)

Random events may occur
 Visitor satisfaction calculated
 Donations collected from happy visitors

EVENING (End of Day)

Revenue calculated (tickets + donations - expenses)
 Unfed animals lose health (-5)
 Dirty enclosures reduce happiness (-10)

Breeding check (healthy pairs may produce offspring)
Visitors leave
Day counter advances

Animal Needs System

Attribute	Range	Decay Rate	Recovery
Health	0-100	-5/day if hungry or dirty	Medicine (+30), Good care (+5/day)
Hunger	0-100	+15/day	Feeding (-50 to -80 depending on food)
Happiness	0-100	-5/day base	Clean enclosure (+10), Companions (+5), Feeding (+5)

Consequences

- **Health < 20** : Animal is SICK - cannot breed, visitors upset
- **Health = 0** : Animal DIES - major visitor satisfaction hit
- **Hunger > 80** : Animal is STARVING - health decreases faster
- **Happiness < 30** : Animal is STRESSED - won't breed, reduced visitor interest

Random Events

Event	Probability	Effect
Heatwave	10%	All animals -10 happiness, water bills +\$200
VIP Visitor	5%	+\$500 donation if satisfaction > 70%
Animal Escape Attempt	3%	Costs \$100 to recapture, -5 visitor satisfaction
Conservation Grant	5%	+\$1000 if animal welfare > 80%
School Excursion	15%	+50% visitors, educational bonus
Media Coverage	5%	If zoo is doing well: +30% visitors next day
Baby Born	Variable	If breeding conditions met

Breeding System

Requirements for breeding:

1. Two animals of same species (different genders if applicable)
2. Both health > 70
3. Both happiness > 60
4. Compatible enclosure (enough space)
5. 10% chance per day if conditions met

7. File Structure

```
ozzoo/  
  main.py                # Entry point, game loop  
  models/  
    __init__.py  
    animal.py            # Animal ABC and hierarchy  
    enclosure.py         # Enclosure ABC and types  
    zoo.py               # Main Zoo class  
    visitor.py           # Visitor class  
    staff.py             # ZooKeeper class  
    resources.py         # Food, Medicine classes  
    interfaces.py        # ICleanable, IFeedable ABCs  
  patterns/  
    __init__.py  
    factory.py           # AnimalFactory  
    observer.py          # Observer pattern (optional)  
  exceptions/  
    __init__.py  
    zoo_exceptions.py    # All custom exceptions  
  ui/  
    __init__.py  
    cli.py               # Command-line interface  
  data/
```

```

    initial_data.json # Starting animals, enclosures (optional)
tests/               # Unit tests (optional)
    test_animals.py

```

For simpler submission, can consolidate to:

```

ozzoo_game/
  ozzoo_main.py          # Everything in one file (or split into 2-3)
  ozzoo_models.py        # All classes
  ozzoo_ui.py            # CLI interface

```

8. Class Count Verification

```

| # | Class | Purpose | Category |
|---|-----|-----|-----|
| 1 | **Animal** | Abstract base for all animals | ABC |
| 2 | **Mammal** | Intermediate class for mammals | Inheritance |
| 3 | **Bird** | Intermediate class for birds | Inheritance |
| 4 | **Reptile** | Intermediate class for reptiles | Inheritance |
| 5 | **Koala** | Specific mammal implementation | Concrete |
| 6 | **Kangaroo** | Specific mammal implementation | Concrete |
| 7 | **Enclosure** | Abstract base for enclosures | ABC |
| 8 | **EucalyptusGrove** | Specific enclosure type | Concrete |
| 9 | **OutbackSavanna** | Specific enclosure type | Concrete |
| 10 | **Zoo** | Main game controller | Core |
| 11 | **Visitor** | Represents zoo visitors | Core |
| 12 | **Food** | Food items for animals | Resource |
| 13 | **Medicine** | Medicine for sick animals | Resource |
| 14 | **ZooKeeper** | Staff member | Core |
| 15 | **AnimalFactory** | Factory pattern | Pattern |
| 16+ | Additional animals (Wombat, Emu, etc.) | Concrete | Extensions |

**Total: 15+ classes** (exceeds minimum of 10)

```

9. Skeleton Code Structure

animal.py (Core)

```

"""
OzZoo Animal Module
Contains Animal ABC and all animal type implementations.
"""

from abc import ABC, abstractmethod
from typing import List, Optional
from interfaces import IFeedable

class Animal(ABC, IFeedable):
    """
    Abstract base class for all zoo animals.

    Attributes:
        name (str): Animal's name
        age (int): Age in years
        health (int): Health level 0-100
        hunger (int): Hunger level 0-100 (higher = hungrier)
        happiness (int): Happiness level 0-100
    """

    # Class constants
    MAX_HEALTH = 100
    MAX_HUNGER = 100
    MAX_HAPPINESS = 100
    HUNGER_THRESHOLD = 80 # Starving
    HEALTH_CRITICAL = 20 # Sick

```

```

def __init__(self, name: str, age: int, species: str):
    """
    Initialize a new animal.

    Args:
        name: Animal's given name
        age: Age in years (must be >= 0)
        species: Species identifier
    """
    self._name = name
    self._age = age
    self._species = species
    self._health = 100
    self._hunger = 30 # Slightly hungry to start
    self._happiness = 70
    self._is_alive = True
    self._observers: List['IObserver'] = []

# Properties with encapsulation
@property
def name(self) -> str:
    """Get animal's name."""
    return self._name

@property
def health(self) -> int:
    """Get current health level."""
    return self._health

@health.setter
def health(self, value: int) -> None:
    """Set health level (clamped to 0-100)."""
    self._health = max(0, min(self.MAX_HEALTH, value))
    if self._health == 0:
        self._is_alive = False
        self._notify_observers("death")
    elif self._health < self.HEALTH_CRITICAL:
        self._notify_observers("sick")

@property
def is_hungry(self) -> bool:
    """Check if animal is hungry (hunger > 50)."""
    return self._hunger > 50

@property
def is_starving(self) -> bool:
    """Check if animal is critically hungry."""
    return self._hunger >= self.HUNGER_THRESHOLD

@property
def is_sick(self) -> bool:
    """Check if animal health is critical."""
    return self._health < self.HEALTH_CRITICAL

# Abstract methods - MUST be overridden
@abstractmethod
def make_sound(self) -> str:
    """Return the sound this animal makes. Must be implemented by subclasses."""
    pass

@abstractmethod
def get_diet(self) -> str:
    """Return the diet type required. Must be implemented by subclasses."""
    pass

@abstractmethod
def get_habitat_type(self) -> str:
    """Return the required habitat type. Must be implemented by subclasses."""
    pass

# Concrete methods
def feed(self, food: 'Food') -> bool:
    """

```


Feed the animal.

Args:

food: The food item to feed

Returns:

bool: True if feeding was successful

"""

```
if food.food_type == self.get_diet():
    self._hunger = max(0, self._hunger - food.nutrition)
    self._happiness = min(self.MAX_HAPPINESS, self._happiness + 5)
    return True
return False
```

```
def daily_update(self) -> None:
```

"""Process daily changes to animal state."""

Hunger increases daily

```
self._hunger = min(self.MAX_HUNGER, self._hunger + 15)
```

Health affected by hunger

if self.is_starving:

```
    self.health -= 10
```

Happiness decay

```
self._happiness = max(0, self._happiness - 5)
```

```
def get_status(self) -> dict:
```

"""Return current status as dictionary."""

return {

"name": self._name,

"species": self._species,

"age": self._age,

"health": self._health,

"hunger": self._hunger,

"happiness": self._happiness,

"is_alive": self._is_alive,

"is_sick": self.is_sick,

"is_hungry": self.is_hungry

}

```
def _notify_observers(self, event: str) -> None:
```

"""Notify all observers of an event."""

for observer in self._observers:

```
    observer.update(self, event)
```

```
def __str__(self) -> str:
```

status = "" if self._health > 50 else "" if self._health > 20 else ""

```
return f"{status} {self._name} ({self._species}) - Health: {self._health}, Hunger: {self._hunger}"
```

```
class Mammal(Animal):
```

"""Intermediate class for mammalian animals."""

```
def __init__(self, name: str, age: int, species: str, fur_color: str = "brown"):
```

```
    super().__init__(name, age, species)
```

```
    self._fur_color = fur_color
```

```
def nurse(self) -> str:
```

"""Mammals can nurse their young."""

```
return f"{self._name} is nursing its young."
```

```
class Bird(Animal):
```

"""Intermediate class for bird animals."""

```
def __init__(self, name: str, age: int, species: str, wingspan: float, can_fly: bool = True):
```

```
    super().__init__(name, age, species)
```

```
    self._wingspan = wingspan
```

```
    self._can_fly = can_fly
```

```
def fly(self) -> str:
```

"""Attempt to fly."""

```
if self._can_fly:
```

```

        return f"{self._name} soars through the air!"
    return f"{self._name} flaps its wings but stays on the ground."

```

```

class Reptile(Animal):
    """Intermediate class for reptile animals."""

    def __init__(self, name: str, age: int, species: str, is_venomous: bool = False):
        super().__init__(name, age, species)
        self._is_venomous = is_venomous

    def bask(self) -> str:
        """Reptiles bask in the sun."""
        self._happiness = min(self.MAX_HAPPINESS, self._happiness + 10)
        return f"{self._name} is basking in the warm sun."

# Specific Australian Animals (Level 3 - inherits from Level 2)

class Koala(Mammal):
    """
    Australian Koala - sleepy eucalyptus eater.
    Inheritance: Animal -> Mammal -> Koala (3 levels)
    """

    def __init__(self, name: str, age: int):
        super().__init__(name, age, species="Koala", fur_color="grey")
        self._sleep_hours = 20 # Koalas sleep a lot!

    def make_sound(self) -> str:
        return f"{self._name} makes a deep bellowing sound!"

    def get_diet(self) -> str:
        return "eucalyptus"

    def get_habitat_type(self) -> str:
        return "eucalyptus_grove"

    def sleep(self) -> str:
        """Koalas are famous for sleeping."""
        self._happiness = min(self.MAX_HAPPINESS, self._happiness + 15)
        return f"{self._name} curls up for a nap (only {self._sleep_hours} hours today!)"

class Kangaroo(Mammal):
    """
    Australian Kangaroo - hopping marsupial.
    Inheritance: Animal -> Mammal -> Kangaroo (3 levels)
    """

    def __init__(self, name: str, age: int, has_joey: bool = False):
        super().__init__(name, age, species="Kangaroo", fur_color="red-brown")
        self._has_joey = has_joey
        self._hop_strength = 80

    def make_sound(self) -> str:
        return f"{self._name} makes a clucking sound!"

    def get_diet(self) -> str:
        return "grass"

    def get_habitat_type(self) -> str:
        return "outback_savanna"

    def hop(self) -> str:
        """Kangaroos love to hop."""
        return f"{self._name} hops around energetically!"

    def box(self) -> str:
        """Kangaroos can box!"""
        return f"{self._name} stands up and boxes the air!"

```

10. CLI Interface Design

Welcome to OZZ00!
Australia's Premier Wildlife Park

Day: 15 Budget: \$12,450 Visitors Today: 127

MAIN MENU

- | | |
|-------------------------|-----------------------------|
| [1] View Zoo Status | [5] Buy Food/Medicine |
| [2] View Animals | [6] Purchase New Animal |
| [3] Feed Animals | [7] Build/Upgrade Enclosure |
| [4] Clean Enclosures | [8] Set Ticket Prices |
| | |
| [9] Advance to Next Day | [0] Save & Quit |

Enter choice: _

Sample Game Flow

=== DAY 1 BEGINS ===

Good morning, Manager! Let's check on OzZoo.

DAILY REPORT:

Animals: 8 (2 sick, 0 critical)
Enclosures: 4 (1 needs cleaning)
Food Stock: Eucalyptus (50), Grass (80), Meat (30)
Budget: \$10,000
Yesterday's Revenue: \$0 (First day!)

ALERTS:

Blinky (Koala) is hungry!
Outback Savanna needs cleaning

What would you like to do?
> 3

=== FEED ANIMALS ===

Which animal to feed?

1. Blinky (Koala) - HUNGRY
 2. Skippy (Kangaroo) - OK
 3. Wally (Wombat) - OK
- > 1

Feeding Blinky...

Blinky happily munches on eucalyptus leaves!
Hunger: 85 35
Happiness: 60 65
Eucalyptus stock: 50 45

11. UML Class Diagram (Text Version)

OzZoo CLASS DIAGRAM

<pre><<interface>> ICleanable + clean() + get_clean() + is_hungry()</pre>	<pre><<interface>> IFeedable + feed() + get_diet()</pre>	<pre><<interface>> IObserver + update()</pre>
<pre><<abstract>> Enclosure 0..* - _id</pre>	<pre><<abstract>> Animal - _name</pre>	

- _name	- _age
- _capacity	- _health
- _animals[]	- _hunger
- _cleanliness	- _happiness
+ add_animal()	+ make_sound()*
+ clean()*	+ eat()
+ get_status()	+ daily_update

Eucalypt Grove	Outback Savanna	Mammal	Bird	Reptile
----------------	-----------------	--------	------	---------

Koala	Kangaroo	Wombat	Emu	SaltwaterCroc
-------	----------	--------	-----	---------------

Zoo	1	*	Visitor	ZooKeeper
- _name			- _id	- _id
- _budget			- _name	- _name
- _enclosures[]			- _ticket_type	- _specialty
- _animals[]			- _satisfaction	
- _visitors[]			+ feed_animal()	
- _day			+ view_animal()	+ clean_encl()
+ donate()				
+ advance_day()				
+ add_animal()				
+ get_report()				

Food

AnimalFactory

- _name	+ create_animal
- _type	+ get_types()
- _cost	
- _nutrition	

Medicine

- _name
- _treats
- _cost

+ apply()

Legend:

Class
Attributes
Methods

Inheritance (is-a)
Composition (has-a)
* Abstract method

12. Implementation Checklist

Must Have (Core Requirements)

- [] 10+ distinct classes
- [] Encapsulation (__protected, __private)
- [] 2-level inheritance hierarchy (Animal Mammal Koala)
- [] Polymorphism (make_sound(), eat() overrides)
- [] At least 1 ABC with abstract methods (Animal)
- [] At least 1 Interface (ICleanable)
- [] At least 1 Design Pattern (Factory)
- [] At least 1 Custom Exception
- [] Game loop (day-by-day)
- [] CLI interface
- [] Resource management (money, food)
- [] Animal welfare system (health, hunger, happiness)

Should Have (Good Grade)

- [] Breeding mechanic
- [] Multiple enclosure types
- [] Random events
- [] Visitor satisfaction system
- [] Medicine/healing system

Nice to Have (Excellence)

- [] Save/load game
- [] Multiple difficulty levels
- [] Achievement system
- [] GUI interface (Tkinter)

13. Estimated Development Time

Component	Estimated Lines	Time
----- ----- -----		
Exceptions	~80	30 min
Interfaces	~50	20 min
Animal hierarchy	~400	2 hours
Enclosure hierarchy	~200	1 hour
Zoo class	~300	1.5 hours
Resources (Food, Medicine)	~100	30 min
Visitor class	~80	30 min
Factory pattern	~80	30 min
CLI interface	~300	2 hours
Game loop & events	~200	1.5 hours
Testing & debugging	-	2 hours
Documentation	-	2 hours
TOTAL	~1800	~14 hours

Design Document Created for NIT2112 OOP Project

OzZoo - Australian Wildlife Park Simulation